

I need to prevent hallucinated locations by forcing the app to only surface stores that can be grounded in real-world evidence (maps, directories, etc.) and by clearly flagging anything that doesn't meet that bar.[redhat+1](#)

## 1. Decide what “real” means in the app

Define hard criteria for a store to be treated as real, for example:

- Must match a result in Google Maps, Apple Maps, or OpenStreetMap at a specific coordinate.
- Or must match at least two independent sources (e.g., a local directory + a news article, or a band council list for native locations + a map marker).[\[tredence\]](#)
- If none of these checks pass, the store stays in an “unverified / AI-suggested” bucket and is not shown as a normal listing.

## 2. Use retrieval instead of pure generation

- Store a vetted corpus: scraped/curated data from known cannabis directories, old forums, archived menus, local news, indigenous business directories, etc.
- Use a retrieval-augmented generation (RAG) pattern: search that corpus for candidates (by name, address, city, coordinates) and only let the LLM talk about locations that retrieval actually found.[redhat+1](#)
- If retrieval returns nothing with sufficient similarity, respond with “no known listing” rather than creating a new one.

## 3. Build a verification pipeline around Maps and directories

For each candidate location your AI proposes:

- Normalize and geocode: send the address/name+city to a geocoding or Places API and require a high-confidence match at the same coordinates.
- Cross-check business category where possible (e.g., “cannabis store”, “smoke shop”), knowing that some legacy or native stores may appear under generic categories but should still resolve to a real POI.[icebergwebdesign+2](#)

- If there is no map POI, try secondary sources (licensed or historic directories, news, municipal enforcement data) before accepting it.

## 4. Scoring and labels for risk

Give each candidate store a confidence score based on:

- Source count and diversity (maps, official registries, news, community lists, user submissions).
- Temporal signals (recent mentions vs. only a single 2017 blog post).
- Model certainty (e.g., embedding similarity between user query and retrieved docs).

Then:

- Only locations above a high threshold enter your main index.
- Show lower-scored entries with explicit labels: “AI-suggested; unverified; may not exist in the real world.”
- Let users filter out AI-suggested locations entirely.

## 5. Strict prompt and output controls

- In prompts, instruct the model to “only return locations that are present in the provided data; if no match exists, answer ‘no known store found’ and do not invent one.”[\[tredence\]](#)
- Use a structured output schema (JSON with fields like `source_ids`, `evidence_count`, `confidence`) and drop any record where `evidence_count == 0`.
- Disallow free-form generation of new addresses or business names; require that all names and addresses come from retrieved records.

## 6. Human-in-the-loop and user feedback

- Give admins a queue of low-confidence stores to manually approve/reject before they go live.

- Let end-users flag “does not exist / wrong location,” and feed that back into your training and scoring logic.
- For sensitive cases (e.g., native land or clearly unregulated shops), consider a policy that nothing is listed without at least one trusted non-AI source plus human review, to avoid safety and legal issues.theglobeandmail+1

## 7. Clear UX to avoid user confusion

- Visually distinguish verified vs. AI-suggested vs. historically closed locations (different badges, colors, or sections).
- Add per-listing disclosure like “Derived from historical/third-party sources; may be inaccurate; not affiliated with Google Business Profile.”
- Provide a detail panel listing the evidence sources (e.g., “Seen in: OpenStreetMap, 2019 CBC article, community report”).[\[tredence\]](#)